

Write Algorithms

Problem 1: Find the Sum of First N Natural Numbers

Algorithm: Sum_Of_N_Numbers

Input: Positive integer N

Output: Sum of first N natural numbers

Step 1: Start.

Step 2: Read N.

Step 3: Set Sum = 0 and I = 1.

Step 4: While $I \leq N$ do

 Sum = Sum + I

 I = I + 1

Step 5: Print Sum.

Step 6: Stop.

Problem 2: Find the Factorial of a Number

Algorithm: Factorial of N

Input: Positive integer N

Output: N!

Step 1: Start.

Step 2: Read N.

Step 3: Set Fact = 1 and I = 1.

Step 4: While $I \leq N$ do

 Fact = Fact $\times$ I

 I = I + 1

Step 5: Print Fact.

Step 6: Stop.

Problem 3: Check Whether a Year is a Leap Year

Algorithm: Leap_Year_Check

Input: Year Y

Output: Leap Year or Not a Leap Year

Step 1: Start.

Step 2: Read Y.

Step 3: If Y is divisible by 4 then

 Step 3.1: If Y is divisible by 100 then

 Step 3.1.1: If Y is divisible by 400 then

 Print "Leap Year"

 Else

 Print "Not a Leap Year"

 Else

 Print "Leap Year"

Else

 Print "Not a Leap Year"

Step 4: Stop.

Problem 4: Loan Eligibility Check

Problem Description

A bank wants to determine whether an applicant is eligible for a personal loan based on the following conditions:

- The applicant must be **at least 21 years old**.
- The applicant must be **60 years old or younger**.
- The applicant must have a **monthly income of ₹30,000 or more**.

If all conditions are satisfied, the applicant is eligible for the loan.

Algorithm: Loan_Eligibility

Input: Age (A) and Monthly Income (I)

Output: Loan Eligible / Not Eligible

Step 1: Start.

Step 2: Read A and I.

Step 3: If $A \geq 21$ then

Step 3.1: If $A \leq 60$ then

Step 3.1.1: If $I \geq 30000$ then

Print "Loan Eligible"

Else

Print "Not Eligible (Insufficient Income)"

Else

Print "Not Eligible (Age Above Limit)"

Else

Print "Not Eligible (Age Below Limit)"

Step 4: Stop.

Problem 5: Scholarship Eligibility

Problem Description

A college offers scholarships based on both academic performance and family income according to the following rules:

- Students scoring **85 marks or above** are considered for a full scholarship.
- To receive a full scholarship, their **family income must not exceed ₹5,00,000 per year**.
- Students scoring **between 75 and 84 marks** receive a partial scholarship.
- Students scoring **below 75 marks** are not eligible for a scholarship.

Algorithm: Scholarship_Eligibility

Input: Marks (M) and Family Income (F)

Output: Scholarship Status

Step 1: Start.

Step 2: Read M and F.

Step 3: If $M \geq 85$ then

Step 3.1: If $F \leq 500000$ then

Print "Full Scholarship"

Else

Print "Merit Certificate"

Else

Step 3.2: If $M \geq 75$ then

Print "Partial Scholarship"

Else

Print "No Scholarship"

Step 4: Stop.

Problem 6: Determine Instagram Creator Badge

Problem Description

Instagram awards creator badges based on the number of followers and engagement rate.

The rules are:

- If a creator has **10,000 or more followers**, they are eligible for badge evaluation.
- Among eligible creators:
 - If engagement rate is **5% or above**, award **Gold Creator Badge**.
 - Otherwise, award **Silver Creator Badge**.
- If a creator has fewer than 10,000 followers:
 - If followers are at least 5,000, award **Rising Creator Badge**.
 - Otherwise, award **No Badge**.

Algorithm: Instagram_Creator_Badge

Input: Number of Followers (F), Engagement Rate (E)

Output: Creator Badge

Step 1: Start.

Step 2: Read F and E.

Step 3: If $F \geq 10000$ then

 Step 3.1: If $E \geq 5$ then

 Print "Gold Creator Badge"

 Else

 Print "Silver Creator Badge"

Else

 Step 3.2: If $F \geq 5000$ then

 Print "Rising Creator Badge"

 Else

 Print "No Badge"

Step 4: Stop.

Finding Time Complexity

Exercise 1: Count Vowels in a String

Problem Statement

Given a string of length n , count the number of vowels (a, e, i, o, u) present in the string.

Algorithm

Read string S

count = 0

For $i = 0$ to $n-1$

 If $S[i]$ is a vowel

 count = count + 1

Print count

Execution Count Table

Step	Statement	Number of Executions
1	Read string S	1
2	count = 0	1
3	For $i = 0$ to $n-1$	$n+1$
4	If $S[i]$ is a vowel	n
5	count = count + 1	$\leq n$
6	Print count	1

$$T(n) = 1 + 1 + (n+1) + n + n + 1 = 3n + 4$$

Time Complexity $O(n)$

Exercise 2: Find Highest Marks in a Class

Problem Statement

Given the marks of n students, find the highest mark scored in the class.

Algorithm

Read marks array

max = marks[0]

For i = 1 to n-1

 If marks[i] > max

 max = marks[i]

Print max

Execution Count Table

Step	Statement	Number of Executions
1	Read marks array	1
2	max = marks[0]	1
3	For i = 1 to n-1	n
4	If marks[i] > max	n-1
5	max = marks[i]	n-1
6	Print max	1

$$T(n) = 1 + 1 + n + (n-1) + (n-1) + 1 = 3n + 1$$

Time Complexity $O(n)$

Exercise 3: Print Multiplication Tables from 1 to n

Problem Statement

Print the multiplication table for every number from 1 to n.

Example for n = 3

1 x 1 = 1

1 x 2 = 2

...

3 x 10 = 30

Algorithm

For i = 1 to n

For j = 1 to 10

Print i * j

Execution Count Table

Step	Statement	Number of Executions
1	For i = 1 to n	n+1
2	For j = 1 to 10	11n
3	Print i * j	10n

$$T(n) = (n+1) + 11n + 10n = 22n + 1$$

Time Complexity $O(n)$

Exercise 4: Generate All Possible Combinations of Three Products

Problem Statement

An online store has n products.

Generate all possible combinations of three products.

Algorithm

For i = 1 to n

For j = 1 to n

For k = 1 to n

Print(i,j,k)

Execution Count Table

Step	Statement	Number of Executions
1	For i = 1 to n	n+1
2	For j = 1 to n	n(n+1)
3	For k = 1 to n	$n^2(n+1)$
4	Print(i,j,k)	n^3

$$T(n) = (n+1) + n(n+1) + n^2(n+1) + n^3 = 2n + 2n^2 + 2n^3 + 1$$

Time Complexity $O(n^3)$

Exercise 5: Find Whether a Student is Enrolled in Each Course

Problem Statement - A college offers **n courses**. For each course, the list of enrolled student IDs is maintained in **sorted order**. Given a student ID, determine whether the student is enrolled in each course using **Binary Search**.

Algorithm

Input: '**N**' courses and each course has '**S**' no of students enrolled , StudentID to search

For each course $i = 1$ to n

 Perform Binary Search on the sorted student list (S students) of course i

 If StudentID found

 Print "Enrolled"

 Else

 Print "Not Enrolled"

Execution Count Table

Step	Statement	Number of Executions
1	For each course $i = 1$ to n	$n+1$
2	Perform Binary Search on the sorted student list (S students) of course i	$n(\log S + 1)$
3	If StudentID found	n
4	Print "Enrolled"	0
5	Else	n
6	Print "Not Enrolled"	n

$T(n) = (n+1) + n(\log S + 1) + n + n + n = 5n + n\log S + 1$; assume $n=S$, then $T(n) = 5n + n\log n + 1$

Time Complexity $O(n\log n)$

Solving Recurrence Relation Equations

Substitution method

1. $T(n) = T(n-1) + 2$; $T(1) = 1$

Solution : Substitute repeatedly:

$$\begin{aligned}T(n) &= T(n-1) + 2 \\&= T(n-2) + 2 + 2 \\&= T(n-3) + 2 + 2 + 2 \\&= T(n-k) + 2k\end{aligned}$$

Let $n-k=1$, $k=n-1$

Substituting:

$$T(n) = T(1) + 2(n-1) = 1 + 2n - 2 = 1 + 2n - 2 = 2n - 1$$

Complexity - $O(n)$

2. $T(n) = T(n/2) + 5$, $T(1) = 1$

Solution :

Substitute repeatedly:

$$\begin{aligned}T(n) &= T(n/2) + 5 \\&= T(n/4) + 5 + 5 \\&= T(n/8) + 5 + 5 + 5 \\&= T(n/2^k) + 5k\end{aligned}$$

When recursion stops , $n/2^k = 1$;
 $n = 2^k$

Taking log on both sides ,
 $\log_2 n = k$

Substituting:

$$T(n) = T(1) + 5\log_2 n = 1 + 5\log_2 n$$

Complexity - $O(\log n)$

3. $T(n) = T(n-1) + 2n$, $T(1) = 1$

Solution :

Substitute repeatedly:

$$\begin{aligned}
 T(n) &= T(n-1) + 2n \\
 &= T(n-2) + 2(n-1) + 2n \\
 &= T(n-3) + 2(n-2) + 2(n-1) + 2n
 \end{aligned}$$

Continuing:

$$T(n) = T(1) + 2(2+3+4+\dots+n)$$

Using

$$2+3+4+\dots+n = \frac{n(n+1)}{2} - 1$$

Therefore

$$\begin{aligned}
 T(n) &= 1 + 2\left(\frac{n(n+1)}{2} - 1\right) \\
 &= 1 + n(n+1) - 2 \\
 &= n^2 + n - 1
 \end{aligned}$$

Complexity - $O(n^2)$

4. $T(n) = T(n/2) + n/2$, $T(1) = 1$

Solution :

Substitute repeatedly:

$$\begin{aligned}
 T(n) &= T(n/2) + n/2 \\
 &= T(n/4) + n/4 + n/2 \\
 &= T(n/8) + n/8 + n/4 + n/2
 \end{aligned}$$

Continuing:

$$T(n) = T(n/2^k) + n\left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{2^k}\right)$$

When recursion stops, $n/2^k = 1$;

$$n = 2^k$$

Taking log on both sides ,

$$\log_2 n = k$$

Therefore substituting

$$T(n) = T(1) + n\left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots\right)$$

Geometric series

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots < 1$$

$$T(n) = T(1) + n = 1 + n$$

Complexity - $O(n)$

5. $T(n) = 2T(n/2) + n$, $T(1) = 1$

Solution :

Substitute repeatedly:

$$\begin{aligned}
 T(n) &= 2T(n/2) + n \\
 &= 2[2T(n/4) + n/2] + n \\
 &= 4T(n/4) + n + n \\
 &= 4T(n/4) + 2n
 \end{aligned}$$

Continuing:

$$=4[2T(n/8)+n/4]+2n$$

$$=8T(n/8)+n+2n$$

$$=8T(n/8)+3n$$

$$T(n)=2^kT(n/2^k)+kn$$

When recursion stops , $n/2^k=1$;

$$n=2^k$$

Taking log on both sides ,

$$\text{Log}_2 n = k$$

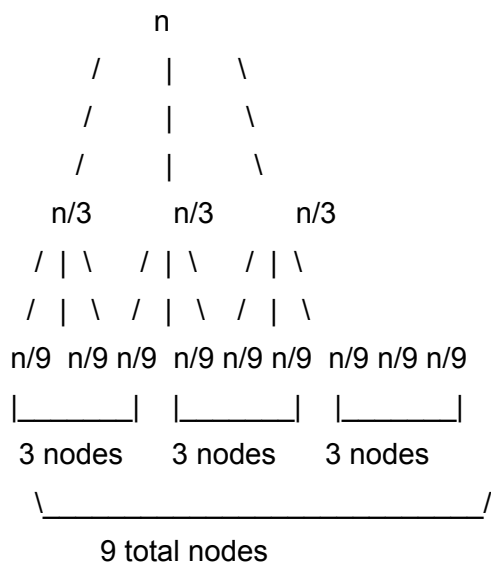
Therefore substituting

$$T(n) = nT(1)+n\text{Log}_2 n$$

Complexity - $O(n\text{Log}_2 n)$

Tree Method

6. $T(n)= 3T(n/3)+n$; $T(1)=1$

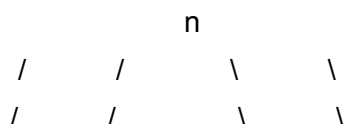


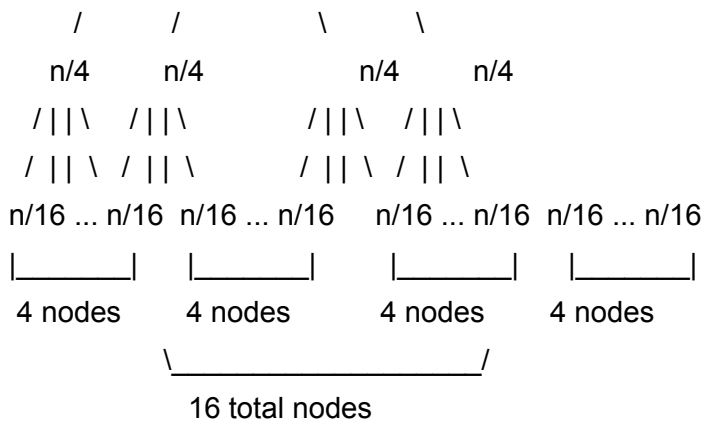
Cost per Level n ; **Number of levels:** $\log_3 n$

Total Cost : $n\log_3 n$

Complexity - $O(n\log n)$

7. $T(n)= 4T(n/4)+n$; $T(1)=1$



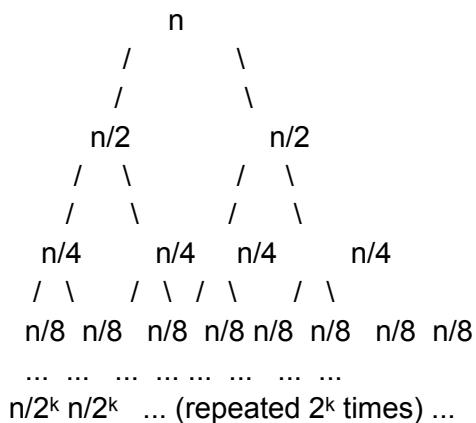


Cost per Level n ; **Number of levels:** $\log_4 n$

Total Cost : $n \log_4 n$

Complexity - $O(n \log n)$

8. $T(n) = 2T(n/2) + n$; $T(1) = 1$



Cost per Level

Level 0 - n

Level 1 - $2 * n/2 = n$

Level 2 - $4 * n/4 = n$

...

:Level k - n

Number of levels - $n/2^k = 1$; $\log_2 n = k$

Total Cost : $(\log_2 n + 1) * n$

Complexity - $O(n \log n)$